

第 2 周学习报告：CS231n Assignment 2

WalkerZJH

2026.05.25 – 2026.05.30

1 任务目标

本阶段围绕 CS231n Assignment 2 展开，目标是完成归一化、dropout、卷积网络和 PyTorch RNN captioning 相关实现，并整理为可复现、便于公开查看的仓库归档。当前目录组织如下：

- 过程代码位于 `units/assignment2/code/`。
- 学习记录和实现记录位于 `units/assignment2/notes/`。
- 实验结果和可视化图片位于 `units/assignment2/results/`。
- 阶段性成果归档位于 `deliverables/week02_assignment2/`。

2 模块架构

当前 Assignment 2 的代码按功能拆分为四层：

- 基础算子层： `cs231n/layers.py` 实现 affine、ReLU、normalization、dropout、convolution、pooling 和 softmax loss。
- 优化器与训练层： `cs231n/optim.py` 和 `cs231n/solver.py` 负责参数更新和训练循环。
- 模型层： `cs231n/classifiers/fc_net.py`、`cnn.py` 和 `rnn_pytorch.py` 实现具体模型。
- 实验归档层： `run_assignment2_experiments.py` 负责统一调度、结果保存和可视化。

3 主要实现

3.1 Batch Normalization 与 Layer Normalization

Batch Normalization 在训练阶段对 batch 内每个特征统计均值和方差：

$$\hat{x} = \frac{x - \mu}{\sqrt{\sigma^2 + \epsilon}}, \quad y = \gamma \hat{x} + \beta$$

并维护 running mean 与 running variance 供测试阶段使用。本次实现同时保留普通 backward 和简化 backward，统一脚本中比较两者输出是否一致。

Layer Normalization 的公式形式相同，但统计维度改为每个样本内部的特征维度，因此训练和测试阶段行为一致，不需要 running statistics。该实现用于小 batch 或序列模型场景下的归一化理解。

3.2 Dropout

Dropout 使用 inverted dropout 形式。训练阶段生成

$$\text{mask} \sim \text{Bernoulli}(p)/p$$

并令输出为 $x \odot \text{mask}$ ；测试阶段直接返回输入。这样输出期望在训练和测试时保持一致，测试阶段不需要额外缩放。

3.3 卷积、池化与快速路径

naive 卷积显式遍历样本、滤波器和空间位置，便于理解 padding、stride 和局部窗口梯度累加。三层 CNN 训练时通过 `fast_layers.py` 调用快速路径。由于本地未编译 Cython 扩展，本次补充了纯 NumPy `im2col/col2im` fallback，使卷积网络实验不依赖本地二进制产物。

Max Pooling 的 forward 在每个局部窗口取最大值，backward 只将梯度传回取得最大值的位置。该逻辑在数值梯度检查中通过。

3.4 Spatial Normalization

Spatial Batch Normalization 将卷积特征图从 (N, C, H, W) 变换为 $(N \cdot H \cdot W, C)$ 后复用普通 BatchNorm。Group Normalization 则将每个样本的通道拆为 G 组，在组内统计均值和方差，不依赖 batch 统计。

3.5 PyTorch RNN/LSTM Captioning

RNN/LSTM 部分使用 PyTorch 张量和 autograd，因此本次补全 forward 路径：图像特征先投影为初始 hidden state，caption token 经 word embedding 后输入 RNN 或 LSTM，每个时间步的 hidden state 再通过 temporal affine 得到词表 score，最后使用 masked temporal softmax 计算 loss。采样阶段从 <START> token 开始逐步生成词 id。

4 方法原理补充

本节补充 Assignment 2 报告中偏理论的部分，重点说明各模块的目标、基本数学形式和 forward/backward 逻辑。

4.1 Batch Normalization

Batch Normalization 解决深层网络训练中各层输入分布持续变化的问题。对一个 batch 中的特征矩阵 $X \in \mathbb{R}^{N \times D}$ ，每个特征独立计算：

$$\mu = \text{mean}(X, \text{axis} = 0)$$

$$\sigma^2 = \text{mean}((X - \mu)^2, \text{axis} = 0)$$

$$\hat{x} = \frac{X - \mu}{\sqrt{\sigma^2 + \epsilon}}$$

$$\text{out} = \gamma \hat{x} + \beta$$

反向传播核心是把上游梯度依次传回 β 、 γ 、归一化结果、方差、均值和输入。简化形式可写为：

$$dx = \frac{1}{N} \cdot \frac{1}{\text{std}} \left(N d\hat{x} - \sum d\hat{x} - \hat{x} \sum (d\hat{x} \odot \hat{x}) \right)$$

naive backward 按计算图逐步展开，便于检查推导；vectorized / alternative backward 将同类项合并，计算更短也更适合工程实现。

4.2 Layer Normalization

Layer Normalization 解决 batch size 较小或序列模型中 batch 统计不稳定的问题。它对每个样本内部的特征维度归一化：

$$\begin{aligned} \mu_i &= \text{mean}(x_i) \\ \sigma_i^2 &= \text{mean}((x_i - \mu_i)^2) \\ \hat{x}_i &= \frac{x_i - \mu_i}{\sqrt{\sigma_i^2 + \epsilon}} \end{aligned}$$

forward 与 BatchNorm 形式接近，但统计轴从 batch 维度换成特征维度。backward 可复用 BatchNorm 的简化公式，只需把 N 换成每个样本的特征数 D ，并按 $\text{axis} = 1$ 求和。

4.3 Dropout

Dropout 通过随机屏蔽隐藏单元缓解共适应和过拟合。当前实现使用 inverted dropout：

$$\begin{aligned} \text{mask} &= \frac{\text{Bernoulli}(p)}{p} \\ \text{out} &= x \odot \text{mask} \end{aligned}$$

训练时 forward 应用 mask，backward 为：

$$dx = dout \odot \text{mask}$$

测试时直接返回输入。与普通 dropout 相比，inverted dropout 把缩放放在训练阶段，测试阶段无需额外处理。

4.4 Convolution

卷积层解决图像局部模式提取问题。对输入 $x \in \mathbb{R}^{N \times C \times H \times W}$ 和滤波器 $w \in \mathbb{R}^{F \times C \times HH \times WW}$ ，输出为：

$$\text{out}_{n,f,i,j} = \sum (\text{window}(x_n, i, j) \odot w_f) + b_f$$

backward 中， db 是所有空间位置上游梯度求和； dw_f 累加每个输入窗口乘以上游梯度； dx 将每个滤波器按上游梯度分配回对应输入窗口。

naive 实现显式遍历样本、滤波器和空间位置，便于理解；vectorized 实现通过 im2col 把卷积转换为矩阵乘法，显著减少 Python 循环开销。

4.5 Max Pooling

Max Pooling 通过局部最大值下采样降低空间分辨率并增强局部平移鲁棒性。forward 对每个 pooling window 取最大值：

$$\text{out}_{n,c,i,j} = \max(\text{window}(x_{n,c}, i, j))$$

backward 只把梯度传给 forward 中取得最大值的位置。naive 实现通过 mask 定位最大值；向量化实现通常借助 reshape 或 im2col 组织窗口。

4.6 Spatial BatchNorm 与 GroupNorm

Spatial BatchNorm 面向卷积特征图，将 (N, C, H, W) 变换为 $(N \cdot H \cdot W, C)$ ，对每个通道做 BatchNorm，再变回原形状。它解决 CNN 中每个通道激活尺度漂移的问题。

GroupNorm 将通道分为 G 组，在每个样本的组内统计均值和方差：

$$x_{\text{group}} \in \mathbb{R}^{N \times G \times (C/G) \times H \times W}$$

GroupNorm 不依赖 batch 维度，因此小 batch 或显存受限时更稳定。它的 backward 与 LayerNorm 类似，只是归一化范围变成每个样本的每个通道组。

4.7 RNN 与 LSTM Captioning

Vanilla RNN 用隐藏状态递推建模序列：

$$h_t = \tanh(x_t W_x + h_{t-1} W_h + b)$$

LSTM 使用输入门、遗忘门、输出门和候选记忆缓解长序列梯度消失：

$$i, f, o = \sigma(a_i), \sigma(a_f), \sigma(a_o)$$

$$g = \tanh(a_g)$$

$$c_t = f \odot c_{t-1} + i \odot g$$

$$h_t = o \odot \tanh(c_t)$$

Captioning 模型中，图像特征先投影成初始 hidden state，caption token 经 embedding 后输入 RNN/LSTM，每个时间步用 temporal affine 输出词表 score，并用 masked temporal softmax 忽略 <NULL> padding。PyTorch 版本由 autograd 处理 backward，本次实现重点是 forward 计算路径和采样流程。

5 实验设置

实验使用 CIFAR-10 子集：

- 训练集：2000 张。
- 验证集：500 张。
- 测试集：500 张。
- 训练轮数：10 epochs。
- batch size：100。
- 运行环境：conda minimind。

统一运行命令为：

```
conda run -n minimind python units\assignment2\code\run_assignment2_experiments.py
```

6 实验结果

模型	训练准确率	验证准确率	测试准确率	最终 loss
FullyConnectedNet + BN + Dropout	0.5680	0.3420	0.3200	2.2012
ThreeLayerConvNet	0.8520	0.4620	0.4000	0.4084

表 1: CIFAR-10 子集实验结果

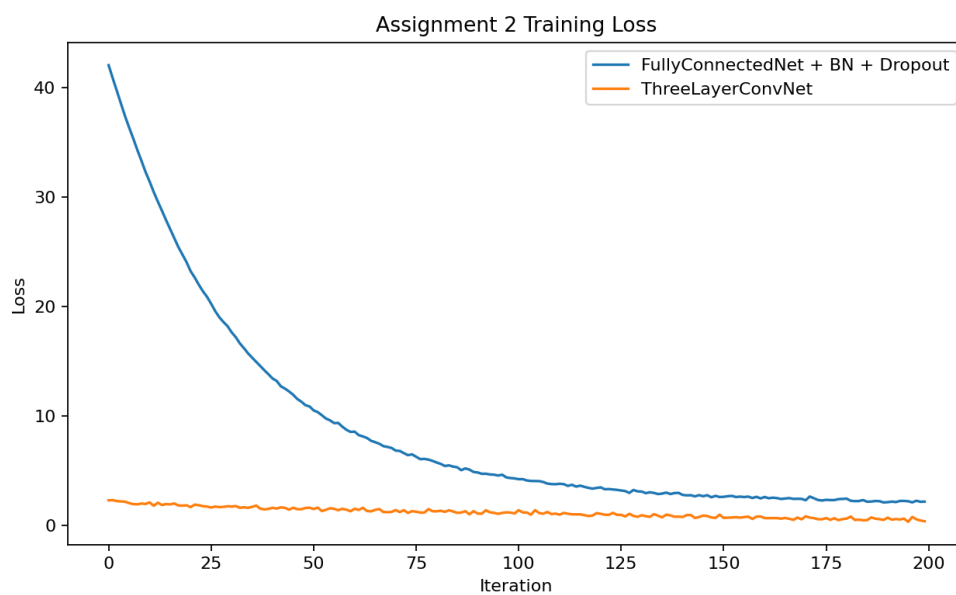


图 1: Assignment 2 训练损失曲线

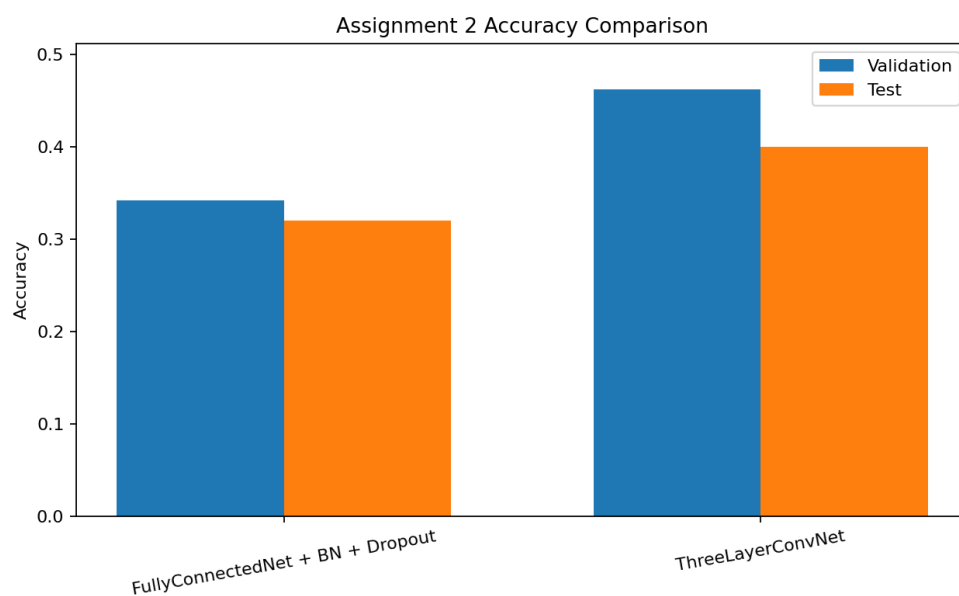


图 2: Assignment 2 验证集与测试集准确率对比

7 结果分析

在当前训练配置下,三层卷积网络的验证准确率和测试准确率均高于加入 BatchNorm 与 Dropout 的全连接网络。这符合图像任务中的基本预期:卷积层利用局部感受野和参数共享,更适合 CIFAR-10 这类具有空间结构的输入。

全连接网络的训练准确率为 0.5680, 验证准确率为 0.3420, 测试准确率为 0.3200, 说明全连接结构在图像空间结构利用上仍然有限。三层 CNN 的训练准确率为 0.8520, 验证准确率为 0.4620, 测试准确率为 0.4000, 训练集和验证集差距提示已经出现一定过拟合, 但相较全连接网络仍有更好的泛化表现。当前结果是本阶段归档配置下的复现实验, 不等同于充分调参后的最终上限。

8 阶段小结

本阶段完成了 Assignment 2 的整理: 补全了核心层、优化器、全连接网络、卷积网络和 PyTorch RNN/LSTM 前向路径, 形成了统一实验脚本, 并保存了指标、日志、摘要和可视化图。通过补充方法原理说明, 本报告进一步覆盖了 BatchNorm、LayerNorm、Dropout、卷积、池化、Spatial Normalization 以及 RNN/LSTM Captioning 的基本动机和计算形式。后续可以在当前结构上继续扩展完整 PyTorch CIFAR-10 训练和 COCO captioning 实验。